

Curating an Analysis-Ready Formula 1 Telemetry Dataset for Reproducible Performance Research

Final Report, IS547 Spring 2026

Author: Anand Marepalli (anandm5@illinois.edu)

Date: 2026-05-06

Github Repo: F1-Telemetry-DE

(<https://github.com/anandm84/F1-telemetry-DE>)

1. Introduction and scenario

For this project, I built and documented an end-to-end curation workflow for public Formula 1 timing data. I treat myself as a data engineer working as a data steward inside a hypothetical university motorsports analytics lab. The lab studies driver consistency, tire degradation, and race strategy, and the immediate use case is to give those researchers a curated, analysis-ready, lap-level dataset (with race-result and weather context) so they don't each have to write their own extraction and cleaning code.

I followed a research-oriented seven-stage lifecycle: Planning, Acquisition, Processing and Quality Assessment, Documentation and Metadata Creation, Storage and Organization, Reproducibility and Transparency, and Preservation and Dissemination. This is consistent with the DCC Curation Lifecycle Model and with the FAIR principles (Wilkinson et al., 2016). Each major section of this report maps to one of these stages.

Two pieces of feedback shaped the final-stage work most. The peer reviewer wrote that the project read "more like a data pipeline than a fully curated dataset" and asked for stronger user-facing documentation, a data dictionary, clearer provenance, and one complete example output. The instructor pushed harder on responsible curation and pointed me to an open question on the FastF1 GitHub discussions ([#634](#)) along with the [Formula1.com Timing Data guidelines](#). He also asked what risks a research lab would face if it adopted this dataset. I addressed both threads directly. The licensing memo, FAIR self-assessment, and risk register are dedicated artifacts in this submission.

2. Dataset and artifact characterization

The dataset comes from public F1 session data accessed through the FastF1 Python library, with schedule and reference data from the Jolpica-F1 API (the successor to the deprecated Ergast service). I ingested the full 2024 season: all 24 rounds, 120 successfully-loaded sessions, covering each round's race and qualifying plus at least one practice session, six sprint races, and six sprint-quali sessions where they exist on the calendar. Data elements include lap timing, sector splits, tire compounds, pit-stop indicators, race results, driver and team metadata, and per-session weather snapshots.

I organized the data in a three-layer medallion architecture, materialized in a single DuckDB warehouse file at `data/warehouse.duckdb`. **Bronze** is append-only NDJSON, partitioned by session sub-folder (`{year}_{round}_{session}`) and, for laps and results, by driver. Each line keeps every field FastF1 returns plus operational lineage (`event_ts` , `_kafka_topic` , `_kafka_offset` , `_kafka_timestamp` , `_bronze_written_at`). **Silver** is dbt-managed tables in the `main_silver` schema, deduplicated on a deterministic SHA-1 `record_id` , with durations parsed from the `HH:MM:SS.NNNN` string encoding into integer milliseconds via the `duration_to_ms` macro and identifiers normalized. **Gold** is dbt-managed tables in the `main_gold` schema, organized as a small star schema: three aggregation tables (`driver_pace` , `tire_performance` , `sector_analysis`), three dimensions (`dim_drivers` , `dim_circuits` , `dim_races`), and three facts (`fact_race_results` , `fact_pit_stops` , `fact_weather_snapshots`). Schedule reference data (`dim_circuits` , `dim_races`) comes from version-controlled dbt seeds at `dbt/seeds/{circuits,races}.csv` , generated by `ingestion/load_schedules.py` .

The full-2024 build produces ~55,300 silver lap rows, ~2,400 race results, ~6,100 pit stops, ~11,500 weather snapshots, and ~2,360 driver-pace gold rows across 33 drivers, 24 circuits, and 24 races. Every column at every layer is documented in [data_dictionary.md](#) and [data_dictionary.csv](#). Each entry has variable name, data type, units, allowed values, source field, transformation, and an interpretive note where the analytical meaning is not obvious. The interpretive notes serve as a small codebook. For example, they explain that `fastest_lap_ms` is a raw minimum that does not exclude in-laps, out-laps, or safety-car laps. That kind of analytical caveat is exactly what the peer reviewer pointed out as missing.

Related artifacts include the source code (Python producers and consumers, the dbt project at `dbt/` , helper scripts), the Airflow DAGs that orchestrate them, the FastF1 cache directory, and the curation documentation under `docs/curation/` .

3. Curation goals

I prioritized the curation goals as follows:

1. **Reproducibility.** A reviewer must be able to rebuild the gold tables from the same upstream source using only the shipped code and docs.
2. **Independent reuse.** A future analyst who never met me must be able to use the dataset correctly, including knowing what *not* to use it for.
3. **Findability and discoverability.** The deposit must be locatable via DOI and indexed by standard aggregators.
4. **Responsible handling.** Redistribution must respect the upstream data's terms, and lab-context risks must be openly addressed.
5. **Quality.** The dataset must be internally consistent: schemas hold, primary keys are unique, value ranges are sensible.

I put reproducibility first because in research analytics, a result you can't reproduce is a result that doesn't exist. Independent reuse is second because curation is the work of making data understandable to people who didn't produce it. Findability is third because a dataset nobody can locate is un reusable, however rich its metadata. I deliberately put responsibility above quality, because the F1 timing source is bound by terms that constrain what can be shipped at all, and getting that wrong invalidates everything downstream. Quality is foundational but it presupposes the four goals above it.

4. Ethical, legal, and policy compliance

This is the area I underestimated in the original plan and where the instructor's feedback most directly shaped the final submission. The full reasoning is in [LICENSING.md](#); a summary follows.

The underlying F1 timing data is the property of Formula One World Championship Ltd. The [Formula1.com](#) Brand Guidelines restrict the use of timing-derived information to personal, non-commercial viewing, and republication, redistribution, or commercial exploitation requires permission from FOM. FastF1 is a reverse-engineered Python client. Its MIT license covers the *library code*, not the underlying data, and the library author has been clear about that distinction in the discussion thread the instructor pointed me to.

Given that, my redistribution decision is:

- **Raw bronze NDJSON for full coverage is not deposited.** A small truncated excerpt (≤ 200 lines per file) is included in the sample bundle for transparency. Anyone who wants the full bronze runs

`ingestion/backfill.py` themselves, and they assume their own compliance posture by doing so.

- **Derived silver and gold tables are deposited under CC-BY-4.0**, with mandatory attribution to FOM (as the underlying data source), FastF1 and Jolpica-F1 (as access tools), and this project (as the curator).
- **Source code is MIT-licensed** separately. The split-license is recorded in `LICENSE` (code) and `LICENSE-DATA.md` (data and documentation), and reflected in the `dc:rights` field of the metadata record.

I enumerate the lab-context risks (takedown exposure, reputational risk to the institution, funding-review friction, derivative-work scope creep) in [RISK_REGISTER.md](#) §5 with concrete mitigations. My position is that academic use of derived statistics and reproducible code is defensible; redistribution of raw timing data is not.

5. Curation workflow

The workflow follows the lifecycle stages from §1 and is shown in [workflow.mmd](#) (which renders to `workflow.png` via `mmdc`). I describe it in five parts.

Acquisition uses two complementary entry points. The streaming Kafka path (`ingestion/producer.py` plus three topic-specific consumers) and the historical backfill path (`ingestion/backfill.py`) target the same bronze NDJSON layout. The backfill carries a resumable manifest (`data/backfill_manifest.jsonl`) so interrupted runs can be retried without duplication. That property turned out to be essential during the full-2024 build, when FastF1's 500-call/hour rate limit forced a multi-pass acquisition. A third entry point, `ingestion/load_schedules.py`, calls FastF1's schedule endpoint and writes version-controlled CSV seeds at `dbt/seeds/`, which decouples the schedule reference data from gold-build time.

Quality assessment and cleaning happens at the silver layer in dbt staging models (`dbt/models/staging/stg_*.sql`) and is enumerated in [CLEANING_LOG.md](#). I followed two principles: (a) preserve information at bronze so nothing is silently dropped before review, and (b) flag, don't drop, at silver, because what counts as an outlier depends on the analytical question. So a safety-car lap is kept and visible as elevated `avg_lap_time_ms` and `lap_std_dev_ms` in `main_gold.driver_pace`, and the data dictionary's interpretive notes warn analysts that `avg_lap_time_ms` is not a "pure pace" measure.

Storage and organization uses the medallion architecture for its lifecycle alignment. Bronze captures *what was acquired*, silver captures *what we believe to be true after cleaning*, and gold captures *what is fit for analysis*. Bronze is NDJSON, which is line-oriented text, language-agnostic, and survives any future deserializer choice. Silver and gold are DuckDB tables in a single warehouse file (`data/warehouse.duckdb`), built and tested by dbt. For the deposited sample bundle I export Parquet

(durable, columnar, listed in the LoC sustainability guidance) and CSV (for users without Parquet tooling).

Workflow automation uses Apache Airflow. Two DAGs (`f1_pipeline` for streaming, `f1_backfill` for historical) orchestrate the producer, consumers, and `dbt build`. The `dbt` project ships a built-in quality contract: `not_null`, `unique`, and `relationships` tests declared in `_stg_models.yml` and `_marts_models.yml`. Running `dbt build` runs all 14 models *and* all 38 tests, and a failing test fails the build.

Provenance is captured at three levels. Per-record lineage is encoded in every bronze NDJSON line: originating Kafka topic, partition, offset, broker timestamp, the wall-clock at which the producer emitted, and the wall-clock at which the bronze writer flushed. Layer-level lineage lives in [PROVENANCE.md](#), which is written for end users rather than engineers and maps every silver and gold column back to a bronze field, an aggregation, or a deterministic computation. Machine-readable lineage is in [prov.jsonld](#), a W3C PROV-O record describing the entities (upstream feed, bronze, silver, gold, deposit), the activities that derived them, and the agents (me, plus FastF1 and Jolpica-F1 as software agents). `dbt` itself emits a graph-level lineage record in `dbt/target/manifest.json` after each build (Buneman et al., 2001; W3C PROV).

6. Metadata and data documentation

My metadata strategy combines three layers.

The **descriptive metadata record** uses Dublin Core in two serializations: XML ([metadata.xml](#)) and JSON ([metadata.json](#)). I chose Dublin Core because it is the lowest-common-denominator standard that Zenodo and OpenAIRE will index, and because no domain-specific motorsport metadata standard exists. Title, creator, subject keywords, description, publisher, dates, type, format, identifier, source, language, references, temporal coverage, spatial coverage, rights, and provenance are all populated explicitly. The DOI field is currently a placeholder, replaced with the real Zenodo DOI before final submission.

The **data dictionary** ([data_dictionary.md](#)) is column-by-column, layer-by-layer, with units, allowed values, source field, and transformation history. A companion CSV ([data_dictionary.csv](#)) makes the same content machine-readable. The interpretive notes section is a small codebook that explains the analytical meaning of derived columns, which is the kind of curatorial documentation a pure schema description misses (Riley, 2017).

The **provenance and cleaning narratives** ([PROVENANCE.md](#) and `CLEANING_LOG.md`) are the human-readable counterpart to the machine-readable PROV-O record. The cleaning log enumerates

every rule applied at every layer with rationale and analytical impact, and includes a section that an automated helper (`scripts/cleaning_log.py`) populates with current row counts after each build by querying the DuckDB warehouse.

7. Quality assessment and cleaning

Concrete cleaning decisions are documented row-for-row in `CLEANING_LOG.md`. The most notable interpretive decisions are: deduplication on a content-addressed `record_id` via dbt's

`qualify row_number() ... = 1` pattern (deterministic, idempotent across reruns, latest-write-wins); preservation of `tire_compound = UNKNOWN` rather than dropping it (the frequency of UNKNOWN is itself a quality signal); and the deliberate non-removal of in-laps, out-laps, and safety-car laps (analyst preference about outliers should not be hard-coded into the dataset). dbt's built-in tests enforce these decisions as a contract: a regression that introduces a duplicate `record_id` or a `dim_drivers` row missing a primary key fails the build.

Quality assessment is therefore three-layered: rule-encoded constraints in dbt staging models (filter and qualify-dedup), assertion-based contract in `_stg_models.yml` / `_marts_models.yml` (38 tests), and prose documentation in `CLEANING_LOG.md`. The three layers reinforce each other. Code expresses what was done, tests verify it stays done, and prose explains why.

One operational decision is documented in the cleaning log. During the full-season build, the original `bronze_read` macro caused DuckDB to OOM when scanning 1300+ NDJSON files because of `union_by_name=true`. I fixed the macro (with an inline comment explaining the rationale) and wrote up the fix as a curation decision in the cleaning log rather than burying it in commit history. That kind of operational note is what distinguishes a "pipeline" from a "curated dataset."

8. Reproducibility and transparency

The reproducibility plan is in [REPRODUCE.md](#). Anyone with Python 3.10+ and internet access can rebuild the curated artifacts from an empty checkout: `pip install -r requirements.txt`, then `python ingestion/backfill.py`, then `python ingestion/load_schedules.py`, then `cd dbt && dbt build`. Because of FastF1's 500-call/hour rate limit, a full-season build takes 30 to 60 minutes, and the backfill manifest makes that resumable. My contract is: pinned dependency versions in `requirements.txt`, the resumable backfill manifest, deterministic `record_id` hashes, idempotent silver dedup via `qualify row_number()`, and version-controlled dbt seeds for the schedule. Non-deterministic columns (the dimensions' `_updated_at` wall-clock fields) are called out so reviewers don't interpret expected drift as a bug (Sandve et al., 2013; Stodden, 2014).

The sample bundle in `docs/curation/sample_output/` is the peer reviewer's "one complete example output." For a single race weekend (2024 round 1, all five sessions), it includes truncated bronze excerpts, full silver Parquet (filtered to that race), full gold Parquet, CSV derivatives of the gold tables, and a sample-bundle README with an example query and expected result.

Integrity is ensured via a SHA-256 manifest written by `scripts/checksums.py`. Running `python scripts/checksums.py --verify` recomputes the manifest and compares.

I also kept `scripts/parity_snapshot.py`, which predates this submission and was originally written to compare a Spark-based reference build against the dbt build. It computes a JSON snapshot of row counts and key aggregates and supports a `compare-duckdb` mode that flags any drift larger than 0.1%. That is a stronger reproducibility signal than row counts alone.

9. Archiving and preservation

The preservation plan is in [PRESERVATION.md](#). I chose Zenodo as the primary archive: it issues a real DOI, is backed by CERN for long-term hosting, integrates with GitHub releases, and offers a 50 GB free quota. IDEALS (UIUC institutional) is a candidate secondary deposit. Format choices follow the LoC sustainability guidance. Parquet is the primary format for the sample-bundle silver/gold exports, CSV is the secondary format for accessibility, NDJSON is used for the small bronze excerpt, Markdown plus PDF is used for documentation, Dublin Core (XML and JSON) for metadata, and JSON-LD for provenance. The DuckDB warehouse file itself is *not* deposited, because that would lock readers to DuckDB; reviewers reproduce it from the deposited code and the live source.

Versioning follows semantic versioning aligned with git tags. Each substantive update is a new Zenodo version with its own DOI, parented to a single dataset-level DOI. A pre-deposit checklist in [PRESERVATION.md](#) governs what must be true before depositing.

10. FAIR self-assessment

I scored the dataset against the 15 FAIR sub-principles in [FAIR_assessment.md](#). Twelve are met, three are partial, and zero are unmet. Two of the three partials (F1 and F4) clear automatically once the Zenodo deposit is live: the placeholder DOI is replaced with the issued DOI, and Zenodo's federation pushes the record to OpenAIRE and DataCite within roughly a day. The remaining gap is R1.3 (community-domain standards), which is a domain-level limitation: there is no canonical motorsport telemetry metadata standard. The FAIR document explains the cheap path to closing R1.3 (Frictionless Data Package or [Schema.org](#) Dataset markup), which I noted as a next-step item.

11. Conclusion: lessons learned and next steps

The biggest lesson is that **engineering can outpace documentation**. A project that looks impressive in code can still fail as a *curated* deliverable. My progress-report draft showed substantial pipeline progress but minimal user-facing curation, and both the peer reviewer and the instructor flagged exactly that gap. The final submission rebalances toward documentation, justification, and responsible-use reasoning. In hindsight, the metadata record, data dictionary, and licensing memo should have started in lockstep with the pipeline rather than being deferred to the last phase.

The second lesson is that **ethical and legal compliance for ambiguously-licensed sources requires concrete decisions, not boilerplate**. My original plan said licensing "will be reviewed." That wasn't enough. The final submission makes a specific decision (ship derived data and code, do not redistribute raw timing) and defends it with a decision matrix and a risk register. The instructor's prompt about lab risk forced that concreteness, and the project is better for it.

The third lesson is about provenance. Per-record `_kafka_*` fields and a written prose log feel sufficient until you sit down to write a machine-readable PROV-O record and realize how much implicit knowledge is in your head. The PROV-O exercise revealed, for example, that `dim_circuits` and `dim_races` are sourced from version-controlled dbt seeds rather than live FastF1 calls at gold-build time. That decoupling improves reproducibility, but it isn't visible from the code alone, and it had to be added to the prose record. dbt's auto-generated `manifest.json` is itself a form of free machine-readable provenance that complements the hand-written log.

A fourth lesson came from running the full-season build. A real pipeline finds real bugs. The DuckDB OOM on `union_by_name=true` in `bronze_read.sql` only manifested at 1300+ files, far past anything I'd run locally. Fixing it is itself a curation decision (preferring a stable schema assumption over defensive schema-merging), and documenting that fix in the cleaning log rather than burying it in a commit message is a deliberate transparency choice.

Next steps: (a) deposit to Zenodo Sandbox to obtain a real DOI and replace the placeholder in the metadata, (b) pin the FastF1 dependency to an exact version (currently it's left flexible), (c) add a Frictionless Data Package or [Schema.org](https://schema.org/) Dataset markup to close the FAIR R1.3 gap, and (d) extend coverage to telemetry channels (speed, throttle, brake, gear) that the producer currently skips. The first three are short-horizon items; the fourth changes the dataset's character meaningfully and should be tracked as a v2.0.

12. References

- Briney, K. (2015). *Data Management for Researchers*. Pelagic Publishing.

- Buneman, P., Khanna, S., & Tan, W.-C. (2001). Why and where: A characterization of data provenance. *Lecture Notes in Computer Science*, 1973, 316-330.
- Carroll, M. W. (2015). Sharing research data and intellectual property law: A primer. *PLoS Biology*, 13(8), e1002235.
- dbt Labs. (n.d.). *dbt documentation*. <https://docs.getdbt.com/>
- Dublin Core Metadata Initiative. (2020). *DCMI Metadata Terms*. <https://www.dublincore.org/specifications/dublin-core/dcmi-terms/>
- DuckDB Foundation. (n.d.). *DuckDB documentation*. <https://duckdb.org/docs/>
- Formula One Management. (n.d.). *Brand guidelines and use of Formula 1 information, Timing Data*. <https://www.formula1.com/en/information/guidelines.4EOKE9RRqevL4niTK9kWyt>
- Library of Congress. (n.d.). *Sustainability of Digital Formats: Apache Parquet*. <https://www.loc.gov/preservation/digital/formats/>
- Oehrly, T. (n.d.). *FastF1*. <https://docs.fastf1.dev/>
- Oehrly, T. (n.d.). *Discussion #634, Permission to use F1 timing data*. <https://github.com/theOehrly/Fast-F1/discussions/634#discussioncomment-10720812>
- Riley, J. (2017). *Understanding Metadata: What is Metadata, and What is it For?* National Information Standards Organization (NISO). <https://www.niso.org/publications/understanding-metadata>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10), e1003285.
- Stodden, V. (2014). The reproducible research movement in statistics. *Statistical Journal of the IAOS*, 30(2), 91-93.
- W3C. (2013). *PROV-O: The PROV Ontology*. <https://www.w3.org/TR/prov-o/>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Zenodo. (n.d.). *Zenodo policies*. <https://about.zenodo.org/policies/>

13. AI Declaration

I used Claude to help with editing, structuring, and proofreading this report and the surrounding curation documents. All design decisions, scoping choices, code, data analysis, and final reasoning are mine.